

GUIA TÉCNICO DE FUNCIONALIDADES E CÓDIGO-FONTE

Computação Gráfica (Unidade 1) e Processamento Digital de Imagens (Unidade 2)

Manual Oficial de Mapeamento: Nomes no Sistema, Funções no Código, Localização na UI e Fundamentos

1. VISÃO GERAL DA ARQUITETURA DOS DOIS PROJETOS

O repositório contém dois sistemas completos, modulares e independentes:

- **Projeto 1 (Unidade 1):** Projeto 1 - Computação Gráfica/Projeto_CG_Final/ — Executável: Projeto1_ComputacaoGrafica.exe. Hub: main_menu.py com módulos de Rasterização Discreta (DDA, Bresenham), Cônicas, Splines Bézier, Transformações Homogêneas 2D/3D, Recortes (Cohen-Sutherland e Sutherland-Hodgman), Viewport e Transformações em PGM.
- **Projeto 2 (PDI):** Projeto - Processamento de Imagens/PI/ — Executável: Projeto Processamento de Imagem.exe. Hub: executar_projeto.py com abas de I/O NetPBM (PGM/PBM), Filtros Espaciais (com exibição simultânea das matrizes G_x e G_y), Operações Pixel a Pixel (Aritméticas e Lógicas), Intensidade/Histograma, Morfologia Matemática e Morfismo Temporal (Delaunay).

2. PROJETO 1 — COMPUTAÇÃO GRÁFICA (UNIDADE 1)

■ Funcionalidade no Sistema: Rasterização de Retas — Algoritmo do Ponto Médio (Bresenham) e DDA

■ Função Oficial no Código: `mod_primitivas/Questao1/core/cg_utils.py -> reta_ponto_medio(x0, y0, x1, y1) & reta_dda(x0, y0, x1, y1)`

■ Onde Clicar na Interface: Menu Principal -> Módulo 1 ("Retas e Cônicas") -> Abas "Reta Ponto Médio" e "Reta DDA".

■ Fundamentação Matemática e Computacional: O DDA calcula incrementos em ponto flutuante ($inc_x = dx/passos$, $inc_y = dy/passos$) e arredonda. O Ponto Médio (Bresenham) utiliza aritmética inteira e variável de decisão $d = 2*dy - dx$: se $d < 0$, avança para Leste (E) com $d += 2*dy$; se $d \geq 0$, avança para Nordeste (NE), incrementa y e faz $d += 2*(dy - dx)$, tratando os 8 oitantes do plano cartesiano.

■ Funcionalidade no Sistema: Rasterização de Cônicas — Circunferência e Elipse do Ponto Médio

■ Função Oficial no Código: `mod_primitivas/Questao1/core/cg_utils.py -> circ_ponto_medio(xc, yc, r) & elipse_ponto_medio(xc, yc, rx, ry)`

■ Onde Clicar na Interface: Menu Principal -> Módulo 1 ("Retas e Cônicas") -> Abas "Circunferência Ponto Médio" e "Elipse Ponto Médio".

■ Fundamentação Matemática e Computacional: A Circunferência usa simetria de 8 oitantes com variável de decisão $d = 1 - r$, gerando 8 pixels espelhados por iteração. A Elipse usa simetria de 4 quadrantes dividida em duas zonas: Região 1 ($2*ry^2*x \leq 2*rx^2*y$) com variação em x e Região 2 ($2*ry^2*x > 2*rx^2*y$) com variação em y .

■ Funcionalidade no Sistema: Splines Paramétricas — Curva de Bézier Cúbica

■ Função Oficial no Código: `mod_primitivas/Questao1/core/cg_utils.py -> spline_bezier_cubica(p0, p1, p2, p3, passos)`

■ Onde Clicar na Interface: Menu Principal -> Módulo 1 ("Retas e Cônicas") -> Aba "Spline Bézier Cúbica".

■ Fundamentação Matemática e Computacional: Calcula a curva contínua no intervalo $t \in [0, 1]$ através dos 4 Polinômios de Bernstein de grau 3: $P(t) = (1-t)^3 \cdot P_0 + 3(1-t)^2 \cdot t \cdot P_1 + 3(1-t) \cdot t^2 \cdot P_2 + t^3 \cdot P_3$, garantindo tangência aos vetores de controle nos extremos e contenção na envoltória convexa.

■ Funcionalidade no Sistema: Transformações Geométricas 2D — Rotação com Pivô Central e Composição de Matrizes

■ Função Oficial no Código: `mod_primitivas/Questao1/apps/transf2d_ui.py -> _dlg_rotacionar() & _dlg_composicao()`

■ Onde Clicar na Interface: Menu Principal -> Módulo 1 -> "Transformações 2D" -> Botões "Rotacionar (Pivô)" e "Compor Transformações 2D".

■ Fundamentação Matemática e Computacional: Calcula o baricentro $(xc, yc) = (\Sigma x/N, \Sigma y/N)$ e aplica $M = T(xc, yc) \cdot R(\theta) \cdot T(-xc, -yc)$, mantendo o objeto no lugar. Na Composição, multiplica sucessivamente as matrizes homogêneas 3x3: $M_{total} = M_n \cdot \dots \cdot M_1$ e aplica aos vértices via $P' = M_{total} \cdot [x, y, 1]^T$.

■ Funcionalidade no Sistema: Recorte de Retas 2D — Algoritmo de Cohen-Sutherland com Animação Contínua

■ Função Oficial no Código: `mod_recortes/Questao2/apps/cohen_sutherland_ui.py -> cohen_sutherland_clip() & _animar_passo()`

■ Onde Clicar na Interface: Menu Principal -> Módulo 2 ("Recortes") -> "Recorte Cohen-Sutherland" -> Botão "Iniciar Animação Contínua".

■ Fundamentação Matemática e Computacional: Classifica os extremos pelos Outcodes [TOP=8, BOTTOM=4, RIGHT=2, LEFT=1]. Se $(c0 \mid c1) == 0$, aceita a reta inteira; se $(c0 \mid c1) != 0$, rejeita. Caso contrário, calcula a interseção analítica da reta com a borda externa ativa: $x = x0 + (y_{borda} - y0)/m$ ou $y = y0 + m*(x_{borda} - x0)$.

■ Funcionalidade no Sistema: Recorte de Polígonos 2D — Algoritmo de Sutherland-Hodgman

■ ■ Função Oficial no Código: [mod_recortes/Questao2/apps/sutherland_hodgman_ui.py](#) -> [sutherland_hodgman_clip\(\)](#)

■ ■ Onde Clicar na Interface: Menu Principal -> Módulo 2 ("Recortes") -> "Recorte Sutherland-Hodgman" -> Botões "Etapa 1 (Esquerda)", "Etapa 2 (Direita)", etc.

■ ■ Fundamentação Matemática e Computacional: Pipeline de 4 estágios cortando contra cada uma das 4 retas de borda. Trata os 4 casos de transição de aresta: **1. In->In:** salva V2; **2. In->Out:** calcula interseção I e salva I; **3. Out->Out:** descarta; **4. Out->In:** calcula interseção I e salva I e V2.

■ Funcionalidade no Sistema: Modelagem 3D, Projeção Isométrica e Mapeamento Janela-Viewport

■ ■ Função Oficial no Código: [mod_3d/Questao3/Questao3..py](#) -> [projection_isometric\(\)](#) & [core/cg_utils.py](#) -> [map_window_to_viewport_rect\(\)](#)

■ ■ Onde Clicar na Interface: Menu Principal -> Módulo 3 ("Cubo 3D e Viewport") -> Painéis de Visualização Original e Viewport.

■ ■ Fundamentação Matemática e Computacional: Aplica matrizes 4x4 em coordenadas homogêneas [x, y, z, 1] (Rx, Ry, Rz, Translação, Escala). A **Projeção Isométrica** converte 3D para 2D rotacionando 45° em Y e ~35.26° em X. O mapeamento afim Janela->Viewport aplica: $xv = Vxmin + (xw - Wxmin) * (Vlarg/Wlarg)$ e $yv = Vymin + (Wymax - yw) * (Valt/Walt)$.

■ Funcionalidade no Sistema: Transformações Afins em Imagens PGM — Mapeamento Inverso

■ ■ Função Oficial no Código: [mod_imagens/Questao4/transformacoes.py](#) -> [aplicar_transformacao_afim_inversa\(\)](#)

■ ■ Onde Clicar na Interface: Menu Principal -> Módulo 4 ("Transformações PGM") -> Carregar imagem PGM e aplicar Rotação/Escala/Cisalhamento.

■ ■ Fundamentação Matemática e Computacional: Evita furos e sobreposições (gaps) varrendo cada pixel de saída (x', y') e buscando o pixel original por $(x, y) = M^{-1} \cdot (x', y')$ com interpolação vizinho mais próximo / bilinear.

3. PROJETO 2 — PROCESSAMENTO DIGITAL DE IMAGENS (PDI)

■ Funcionalidade no Sistema: I/O de Imagens — Parser e Gravador NetPBM Estrito (P1, P2, P5)

■ Função Oficial no Código: [src/laboratorio_imagens/core/io_netpbm.py](#) -> [carregar_imagem\(\)](#) & [salvar_imagem\(\)](#)

■ Onde Clicar na Interface: Aba "Filtros Espaciais" ou "Carregar Imagem" -> Diálogo de seleção de arquivos *.pgm e *.pbm.

■ Fundamentação Matemática e Computacional: Leitor e gravador de formatos NetPBM: **P1** (PBM binário preto/branco em ASCII), **P2** (PGM em escala de cinza em texto ASCII) e **P5** (PGM em escala de cinza em bytes brutos). Valida cabeçalhos e armazena em matriz NumPy uint8 [0..255].

■ Funcionalidade no Sistema: Filtros Espaciais — Operadores de Gradiente (Roberts, Roberts Cruzado, Prewitt, Sobel)

■ Função Oficial no Código: [src/laboratorio_imagens/core/filtros_espaciais.py](#) -> [operador_roberts\(\)](#), [operador_roberts_cruzado\(\)](#), [operador_prewitt\(\)](#), [operador_sobel\(\)](#)

■ Onde Clicar na Interface: Aba "Filtros Espaciais" -> Selecionar "Roberts", "Roberts cruzado", "Prewitt" ou "Sobel" -> Exibição simultânea de Gx e Gy.

■ Fundamentação Matemática e Computacional: Calcula derivadas direcionais de 1ª ordem com convolução 2D. O sistema exibe lado a lado as duas matrizes: **Gx** (diferença vertical/diagonal principal) e **Gy** (diferença horizontal/diagonal secundária). A imagem final combina a magnitude: $|Gx| + |Gy|$.

■ Funcionalidade no Sistema: Filtros Espaciais — Média Linear, Mediana Não-Linear e High-Boost

■ Função Oficial no Código: [src/laboratorio_imagens/core/filtros_espaciais.py](#) -> [filtro_media\(\)](#), [filtro_mediana\(\)](#), [filtragem_high_boost\(\)](#)

■ Onde Clicar na Interface: Aba "Filtros Espaciais" -> Selecionar "Filtro da media", "Filtro da mediana" ou "High-boost" (com entrada do Fator A).

■ Fundamentação Matemática e Computacional: A **Média** aplica convolução linear com kernel 1/9. A **Mediana** é não-linear, ordenando os 9 vizinhos e selecionando o elemento central (ideal para ruído Sal e Pimenta). O **High-Boost** realça altas frequências pela fórmula: $\text{HighBoost} = A \cdot \text{Original} - \text{PassaBaixas} = (A-1) \cdot \text{Original} + \text{PassaAltas}$.

■ Funcionalidade no Sistema: Operações Pixel a Pixel — Aritméticas e Lógicas (Truncamento e Normalização)

■ Função Oficial no Código: [src/laboratorio_imagens/core/operacoes_pixel.py](#) -> [soma\(\)](#), [subtracao\(\)](#), [multiplicacao\(\)](#), [divisao\(\)](#), [operacao_and\(\)](#), [operacao_or\(\)](#), [operacao_xor\(\)](#), [operacao_not\(\)](#)

■ Onde Clicar na Interface: Aba "Operações Pixel a Pixel" -> Carregar Imagem A e B -> Selecionar Operação (Soma, Subtração, AND, OR, XOR, NOT).

■ Fundamentação Matemática e Computacional: Ajusta dimensões por interpolação bilinear e aplica operações ponto a ponto. Operações aritméticas tratam overflow/underflow via **Truncamento**: $\text{clip}(A \pm B, 0, 255)$ ou **Normalização**: $((I - \min)/(max - \min)) * 255$. Operações lógicas operam bit a bit sobre uint8 para mascaramento (AND), união (OR) e detecção de diferenças (XOR).

■ Funcionalidade no Sistema: Intensidade e Histograma — Transformações Não-Lineares e Equalização via CDF

■ Função Oficial no Código: [src/laboratorio_imagens/core/transformacoes_intensidade.py](#) & [histograma.py](#) -> [equalizar_histograma\(\)](#)

■ Onde Clicar na Interface: Aba "Intensidade e Histograma" -> Selecionar "Equalize o histograma" ou transformações Gamma, Logaritmo e Sigmóide.

■ Fundamentação Matemática e Computacional: A **Equalização** redistribui os níveis de cinza tornando a Função de Distribuição Acumulada (CDF) uniforme: calcula a probabilidade $p(r_k) = nk / N$, a acumulada $CDF(k) = \sum p(r_j)$ e mapeia cada pixel antigo r_k para o novo s_k por: $s_k = \text{round}(255 \cdot CDF(k))$.

■ Funcionalidade no Sistema: Morfologia Matemática — Erosão, Dilatação, Abertura, Fechamento e Hit-or-Miss

■ Função Oficial no Código: [src/laboratorio_imagens/core/operacoes_morfologicas.py](#) -> [erosao\(\)](#), [dilatacao\(\)](#), [abertura\(\)](#), [fechamento\(\)](#), [hit_or_miss\(\)](#)

■ Onde Clicar na Interface: Aba "Morfologia" -> Selecionar operador e elemento estruturante 3x3 (Cruz ou Quadrado).

■ Fundamentação Matemática e Computacional: Aplica elemento estruturante B. **Erosão**: mínimo local ($A \blacksquare B$); **Dilatação**: máximo local ($A \oplus B$); **Abertura**: $(A \blacksquare B) \oplus B$ (remove ruídos claros sem alterar tamanho); **Fechamento**: $(A \oplus B) \blacksquare B$ (fecha furos escuros); **Hit-or-Miss**: $(A \blacksquare B1) \cap (A^c \blacksquare B2)$ para detecção de configurações exatas.

■ Funcionalidade no Sistema: Morfismo Temporal (Morphing) — Triangulação de Delaunay e Coordenadas Baricêntricas

■ Função Oficial no Código: [src/laboratorio_imagens/core/morfismo.py](#) & [exemplos_morfismo.py](#) -> [preparar_morfismo\(\)](#) & [gerar_frame_preparado\(\)](#)

■ ■ **Onde Clicar na Interface:** Aba "Morfismo Temporal" -> Botão "Carregar caso teste de morfismo" (fotos da criança e adulto) -> Mover slider de tempo $t \in [0, 1]$.

■ **Fundamentação Matemática e Computacional:** Aplica **Triangulação de Delaunay** sobre 13 pontos anatômicos faciais. Para cada instante t : interpola vértices $P(t) = (1-t)P_{ini} + t \cdot P_{fin}$, deforma cada triângulo por **Coordenadas Baricêntricas** (mapeamento afim inverso) e realiza a fusão de cores (Cross-Dissolve): $I(t) = (1-t) \cdot I_{def_ini} + t \cdot I_{def_fin}$.

4. TABELA RÁPIDA DE CONSULTA DAS FUNCIONALIDADES DO SISTEMA

Funcionalidade no Sistema	Função / Arquivo de Código Fonte	Onde Clicar na Interface
Reta DDA e Ponto Médio	reta_ponto_medio / mod_primitivas/.../cg_utils.py	Projeto 1 -> Módulo 1 -> Reta DDA / Ponto Médio
Círculo e Elipse Ponto Médio	circ_ponto_medio, elipse_ponto_medio / cg_utils.py	Projeto 1 -> Módulo 1 -> Círculo / Elipse
Splines Bézier Cúbicas	spline_bezier_cubica / mod_primitivas/.../cg_utils.py	Projeto 1 -> Módulo 1 -> Spline Bézier
Rotação com Pivô Central 2D	_dlg_rotacionar / mod_primitivas/.../transf2d_ui.py	Projeto 1 -> Módulo 1 -> Transf 2D -> Rotacionar (Pivô)
Composição de Transformações 2D	_dlg_composicao / mod_primitivas/.../transf2d_ui.py	Projeto 1 -> Módulo 1 -> Transf 2D -> Compor Transformações
Cohen-Sutherland (Outcodes)	cohen_sutherland_clip / mod_recortes/.../cohen_sutherland_ui.py	Projeto 1 -> Módulo 2 -> Recorte Cohen-Sutherland
Sutherland-Hodgman Polígonos	sutherland_hodgman_clip / mod_recortes/.../sutherland_hodgman_ui.py	Projeto 1 -> Módulo 2 -> Recorte Sutherland-Hodgman
Cubo 3D e Viewport	projection_isometric, map_window_to_viewport / Questao3..py	Projeto 1 -> Módulo 3 -> Cubo 3D e Viewport
Transformação Afim PGM (Inversa)	aplicar_transformacao_afim_inversa / transformacoes.py	Projeto 1 -> Módulo 4 -> Transformações PGM
I/O NetPBM (P1, P2, P5)	carregar_imagem, salvar_imagem / io_netpbm.py	Projeto 2 -> Carregar imagem .pgm / .pbm
Filtros Espaciais (Gx e Gy Duplos)	operador_roberts, operador_sobel / filtros_espaciais.py	Projeto 2 -> Filtros Espaciais (Exibição Dupla Gx e Gy)
Filtro da Média, Mediana e High-Boost	filtro_media, filtro_mediana, filtragem_high_boost	Projeto 2 -> Filtros Espaciais
Operações Aritméticas e Lógicas	soma, subtracao, operacao_and, operacao_xor / operacoes_pixel.py	Projeto 2 -> Operações Pixel a Pixel
Equalização de Histograma (CDF)	equalizar_histograma / histograma.py	Projeto 2 -> Intensidade e Histograma
Morfologia Matemática	erosao, dilatacao, abertura, fechamento / operacoes_morfologicas.py	Projeto 2 -> Morfologia Matemática
Morfismo Temporal (Delaunay)	preparar_morfismo, gerar_frame_preparado / morfismo.py	Projeto 2 -> Morfismo Temporal (Carregar Teste)